SISTEMA DE CALCULADORA CON DESHACER

Patrón de Diseño: *Command*

Desacoplamiento de solicitudes aritméticas encapsulando operaciones como objetos con capacidad de reversión

Autor: **Angie Nathaly Cisneros Guerrero**

20242020084

Fecha: Octubre 2026

Índice

1. Introducción **2**

2. Objetivos **3**
2.1. Objetivo General 3
2.2. Objetivos Específicos 3

3. Diagrama UML **4**

4. Explicación de las Clases **5**
4.1. Receptor (Calculadora.java) 5
4.2. Interfaz Comando (Operacion.java) 6
4.3. Clase Invocadora (Invoker.java) 7
4.4. Comandos Concretos 8
4.4.1. Operacion Sumar 8
4.4.2. Operacion Restar 8
4.4.3. Operacion Multiplicar 9
4.4.4. Operacion Dividir 10
4.4.5. Operacion Googol 10

5. Código de Cliente **12**
5.1. Clase Cliente interactiva (Cliente.java) 12

6. Pruebas y Ejecución **16**
6.1. Prueba Cliente 16
6.2. Pruebas Unitarias 16

7. Repositorio **18**
7.1. Enlace GitHub 18
7.2. Estructura de Paquetes 18

Introducción

En los sistemas de calculadora, la ejecución de operaciones de gran escala (como el cálculo de un Googol) exige un mecanismo estructurado para separar la lógica de la interfaz, desacoplar las peticiones y permitir la reversión de operaciones (*Undo*) sin comprometer el encapsulamiento ni la precisión de los números.

Para dar solución, se implementa el patrón de diseño comportamental **Command**. Este patrón encapsula una solicitud como un objeto, permitiendo parametrizar a los clientes con diferentes peticiones, registrar operaciones y soportar de forma limpia la funcionalidad de deshacer mediante el respaldo del estado anterior en cada comando concreto.

Objetivos

Objetivo General

Implementar una calculadora orientada a objetos aplicando el patrón de diseño **Command** para gestionar la ejecución de operaciones elementales y la operación especial Googol, incorporando un mecanismo de reversión.

Objetivos Específicos

- Diseñar la clase `Calculadora` como receptor (*Receiver*) para centralizar la lógica matemática
- Establecer una interfaz común `Operacion` que unifique los métodos de ejecución y reversión de comandos
- Desarrollar clases concretas para cada operación aritmética y especial, encapsulando el estado anterior.
- Construir la clase `Invoker` para administrar la invocación de las acciones y el control del estado de reversión.
- Diseñar una clase `Cliente` interactiva por consola con soporte para manejo de errores, visualización dinámica de operadores y validación mediante pruebas unitarias independientes.

Diagrama UML

Descripción General

El proyecto implementa una arquitectura basada en el patrón comportamental **Command** (Comando).

Encapsula cada una de las operaciones matemáticas de una calculadora (`OperacionSumar`, `OperacionRestar`, `OperacionMultiplicar`, `OperacionDividir` y `OperacionGoogol`) como objetos individuales que implementan la interfaz común `Operacion`.

Diagrama de Clases

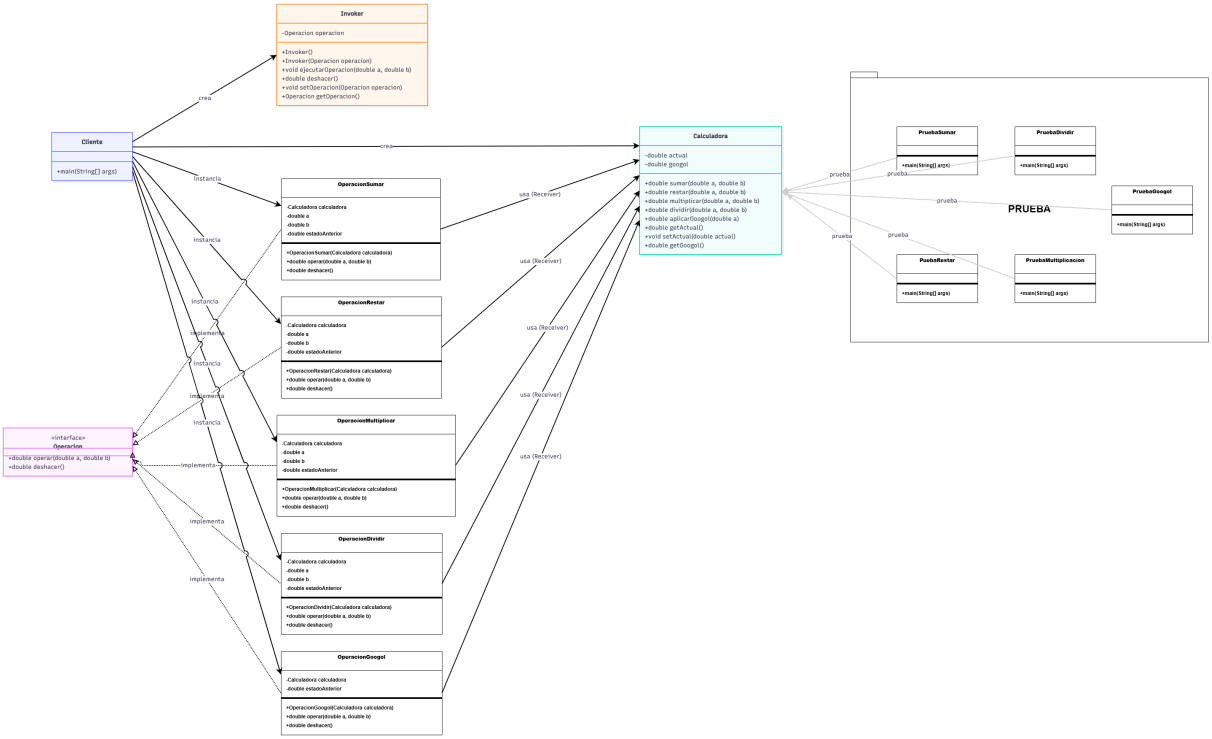


Figura 1: Diagrama de Clases del Patrón Command.

Explicación de las Clases

Receptor (Calculadora.java)

Descripción

Actúa como la clase Receptora (*Receiver*) en la arquitectura Command. Posee el estado actual de la calculadora (`actual`) y la constante de la constante de Googol (10^{100}). Contiene los métodos de bajo nivel para realizar los cálculos matemáticos reales.

```java
package operaciones.command;

public class Calculadora {

    private double actual = 0;
    private final double googol = Math.pow(10, 100);

    public double sumar(double a, double b) {
        this.actual = a + b;
        return this.actual;
    }

    public double restar(double a, double b) {
        this.actual = a - b;
        return this.actual;
    }

    public double multiplicar(double a, double b) {
        this.actual = a * b;
        return this.actual;
    }

    public double dividir(double a, double b) {
        if (b != 0) {
            this.actual = a / b;
            return this.actual;
        }
        throw new IllegalArgumentException("El divisor no puede ser 0");
    }

    public double aplicarGoogol(double a) {
        this.actual = a * googol;
        return this.actual;
    }

    public double getActual() {
        return actual;
    }

    public void setActual(double actual) {
        this.actual = actual;
    }
```

```
43
44      public double getGoogol() {
45          return googol;
46      }
47  }
```

Interfaz Comando (Operacion.java)

Descripción

Declara la interfaz unificada para la ejecución de todos los comandos de la calculadora. Establece el contrato mediante los métodos `operar(a, b)` para aplicar la operación y `deshacer()` para revertir sus efectos sobre el receptor.

```
1  package operaciones.command;
2
3  public interface Operacion {
4      double operar(double a, double b);
5      double deshacer();
6  }
```

Clase Invocadora (Invoker.java)

Descripción

Mantiene la referencia hacia la última operación realizada y actúa como el intermediario entre la interfaz de usuario y los comandos concretos. Dispara la ejecución del método `deshacer()` cuando el usuario lo requiere.

```java
package operaciones.command;

public class Invoker {

    private Operacion operacion;

    public Invoker() {
    }

    public Invoker(Operacion operacion) {
        this.operacion = operacion;
    }

    public void ejecutarOperacion(double a, double b){
        operacion.operar(a,b);
    }

    public double deshacer() {
        if (operacion != null) {
            double resultado = operacion.deshacer();
            operacion = null;
            return resultado;
        }
        System.out.println("No hay ninguna operacion para
           deshacer.");
        return 0;
    }

    public void setOperacion(Operacion operacion) {
        this.operacion = operacion;
    }

    public Operacion getOperacion() {
        return operacion;
    }
}
```

Comandos Concretos

Descripción

Implementaciones concretas de la interfaz `Operacion`. Cada clase vincula al receptor `Calculadora`, captura el estado previo antes de efectuar el cálculo (`estadoAnterior = calculadora.getActual()`) y restaura dicho valor al invocar `deshacer()`.

4.4.1. Operacion Sumar

```
public class OperacionSumar implements Operacion{

    private Calculadora calculadora;
    private double a;
    private double b;
    private double estadoAnterior;

    //Constructor

    public OperacionSumar(Calculadora calculadora) {
            this.calculadora = calculadora;
     }


@Override
    public double operar(double a, double b) {
        this.a = a;
        this.b = b;
        this.estadoAnterior = calculadora.getActual();
        return calculadora.sumar(a, b);
    }

@Override
    public double deshacer() {
        calculadora.setActual(estadoAnterior);
        return calculadora.getActual();
    }

}
```

4.4.2. Operacion Restar

```
public class OperacionRestar implements Operacion{

    private Calculadora calculadora;
    private double a;
    private double b;
    private double estadoAnterior;

    //Constructor

```

```
    public OperacionRestar(Calculadora calculadora) {
            this.calculadora = calculadora;
     }


@Override
    public double operar(double a, double b) {
        this.a = a;
        this.b = b;
        this.estadoAnterior = calculadora.getActual();
        return calculadora.restar(a, b);
    }

@Override
    public double deshacer() {
        calculadora.setActual(estadoAnterior);
        return calculadora.getActual();
    }
}
```

4.4.3. Operacion Multiplicar

```
public class OperacionMultiplicar implements Operacion{

    private Calculadora calculadora;
    private double a;
    private double b;
    private double estadoAnterior;

    //Constructor

    public OperacionMultiplicar(Calculadora calculadora) {
            this.calculadora = calculadora;
     }


@Override
    public double operar(double a, double b) {
        this.a = a;
        this.b = b;
        this.estadoAnterior = calculadora.getActual();
        return calculadora.multiplicar(a, b);
    }

@Override
    public double deshacer() {
        calculadora.setActual(estadoAnterior);
        return calculadora.getActual();
    }


```

```
}
```

4.4.4. Operacion Dividir

```
package operaciones.command;

public class OperacionDividir implements Operacion {

    private Calculadora calculadora;
    private double a;
    private double b;
    private double estadoAnterior;

    public OperacionDividir(Calculadora calculadora) {
        this.calculadora = calculadora;
    }

    @Override
    public double operar(double a, double b) {
        this.a = a;
        this.b = b;
        this.estadoAnterior = calculadora.getActual();
        return calculadora.dividir(a, b);
    }

    @Override
    public double deshacer() {
        calculadora.setActual(estadoAnterior);
        return calculadora.getActual();
    }
}
```

4.4.5. Operacion Googol

```
package operaciones.command;

public class OperacionGoogol implements Operacion {

    private Calculadora calculadora;
    private double estadoAnterior;

    public OperacionGoogol(Calculadora calculadora) {
        this.calculadora = calculadora;
    }

    @Override
    public double operar(double a, double b) {
        this.estadoAnterior = calculadora.getActual();
        return calculadora.aplicarGoogol(a);
```

```
16      }
17
18      @Override
19      public double deshacer() {
20          calculadora.setActual(estadoAnterior);
21          return calculadora.getActual();
22      }
23  }
```

Código de Cliente

Clase Cliente interactiva (Cliente.java)

Descripción

Actúa como punto de entrada de la aplicación. Crea las instancias del receptor, los comandos y el invocador. Gestiona la interacción en consola con el usuario mostrando la expresión dinámica (por ejemplo `4.0 + :`) y asigna los comandos correspondientes al invocador.

Código Fuente: Cliente.java

```
1  package operaciones.command;
2
3  import java.util.InputMismatchException;
4  import java.util.Scanner;
5
6  public class Cliente {
7
8      public static void main(String[] args) {
9
10         Invoker objInvoker = new Invoker();
11         Calculadora objCalculadora = new Calculadora();
12
13         OperacionSumar objSumar = new OperacionSumar(
               objCalculadora);
14         OperacionRestar objRestar = new OperacionRestar(
               objCalculadora);
15         OperacionMultiplicar objMultiplicar = new
               OperacionMultiplicar(objCalculadora);
16         OperacionDividir objDividir = new OperacionDividir(
               objCalculadora);
17         OperacionGoogol objGoogol = new OperacionGoogol(
               objCalculadora);
18
19         Scanner sc = new Scanner(System.in);
20         boolean salir = false;
21
22         while (!salir) {
23             System.out.println("\n------------ Calculadora
                   ------------");
24             System.out.println("----- Valor actual: " +
                   objCalculadora.getActual() + " -----");
25             System.out.println("Operaciones:");
26             System.out.println("1. Sumar");
27             System.out.println("2. Restar");
28             System.out.println("3. Multiplicar");
29             System.out.println("4. Dividir");
30             System.out.println("5. Utilizar Googol");
31             System.out.println("6. Deshacer operacion");
32             System.out.println("7. Salir");
33             System.out.print("Ingrese la opcion que quiere: ");
```

```
34
35              int opcion = 0;
36
37              try {
38                  opcion = sc.nextInt();
39              } catch (InputMismatchException e) {
40                  System.out.println("Error: numero no valido.");
41                  sc.nextLine();
42                  continue;
43              }
44
45              if (opcion == 7) {
46                  salir = true;
47                  System.out.println("Saliendo ....");
48                  break;
49              }
50
51              double a = objCalculadora.getActual();
52              double b = 0;
53              double resultado = 0;
54              String simbolo = "";
55
56              switch (opcion) {
57                  case 1: simbolo = "+"; break;
58                  case 2: simbolo = "-"; break;
59                  case 3: simbolo = "*"; break;
60                  case 4: simbolo = "/"; break;
61              }
62
63              if (opcion >= 1 && opcion <= 4) {
64                  System.out.print(a + " " + simbolo + " :");
65                  b = sc.nextDouble();
66              } else if (opcion == 5) {
67                  a = objCalculadora.getActual();
68              }
69
70              switch (opcion) {
71                  case 1:
72                      resultado = objSumar.operar(a, b);
73                      objInvoker.setOperacion(objSumar);
74                      System.out.println("\n
                            ------------------------------------------
                            ");
75                      System.out.println("------------ " + a + " +
                            " + b + " = " + resultado + "
                            ------------");
76                      System.out.println("
                            ------------------------------------------\
                            n");
77                      break;
78
```

```
79                  case 2:
80                      resultado = objRestar.operar(a, b);
81                      objInvoker.setOperacion(objRestar);
82                      System.out.println("\n
                            ------------------------------------------
                            ");
83                      System.out.println("-------------␣" + a + "␣-
                            ␣" + b + "␣=␣" + resultado + "␣
                            -------------");
84                      System.out.println("
                            ------------------------------------------\
                            n");
85                      break;
86
87                  case 3:
88                      resultado = objMultiplicar.operar(a, b);
89                      objInvoker.setOperacion(objMultiplicar);
90                      System.out.println("\n
                            ------------------------------------------
                            ");
91                      System.out.println("-------------␣" + a + "␣*
                            ␣" + b + "␣=␣" + resultado + "␣
                            -------------");
92                      System.out.println("
                            ------------------------------------------\
                            n");
93                      break;
94
95                  case 4:
96                      try {
97                          resultado = objDividir.operar(a, b);
98                          objInvoker.setOperacion(objDividir);
99                          System.out.println("\n
                                ------------------------------------------
                                ");
100                         System.out.println("-------------␣" + a +
                                "␣/␣" + b + "␣=␣" + resultado + "␣
                                -------------");
101                         System.out.println("
                                ------------------------------------------\
                                n");
102                     } catch (IllegalArgumentException e) {
103                         System.out.println("Error:␣" + e.
                                getMessage());
104                     }
105                     break;
106
107                 case 5:
108                     resultado = objGoogol.operar(a, 0);
109                     objInvoker.setOperacion(objGoogol);
110                     System.out.println("\n
```

```
                        ------------------------------------------
                        ");
111                     System.out.println("-------------␣" + a + "␣*
                        ␣Googol␣=␣" + resultado + "␣-------------")
                        ;
112                     System.out.println("
                        ------------------------------------------\
                        n");
113                     break;
114
115                 case 6:
116                     resultado = objInvoker.deshacer();
117                     System.out.println("\n
                        ------------------------------------------
                        ");
118                     System.out.println("-------------␣Deshecho␣->
                        ␣Estado␣actual:␣" + resultado + "␣
                        -------------");
119                     System.out.println("
                        ------------------------------------------\
                        n");
120                     break;
121
122                 default:
123                     System.out.println("Opcion␣invalida.");
124                     break;
125             }
126         }
127
128         sc.close();
129     }
130 }
```

Pruebas y Ejecución

Prueba Cliente

Salida Consola donde se realiza una suma y posteriormente se utiliza la opción de deshacer para restaurar el estado anterior:

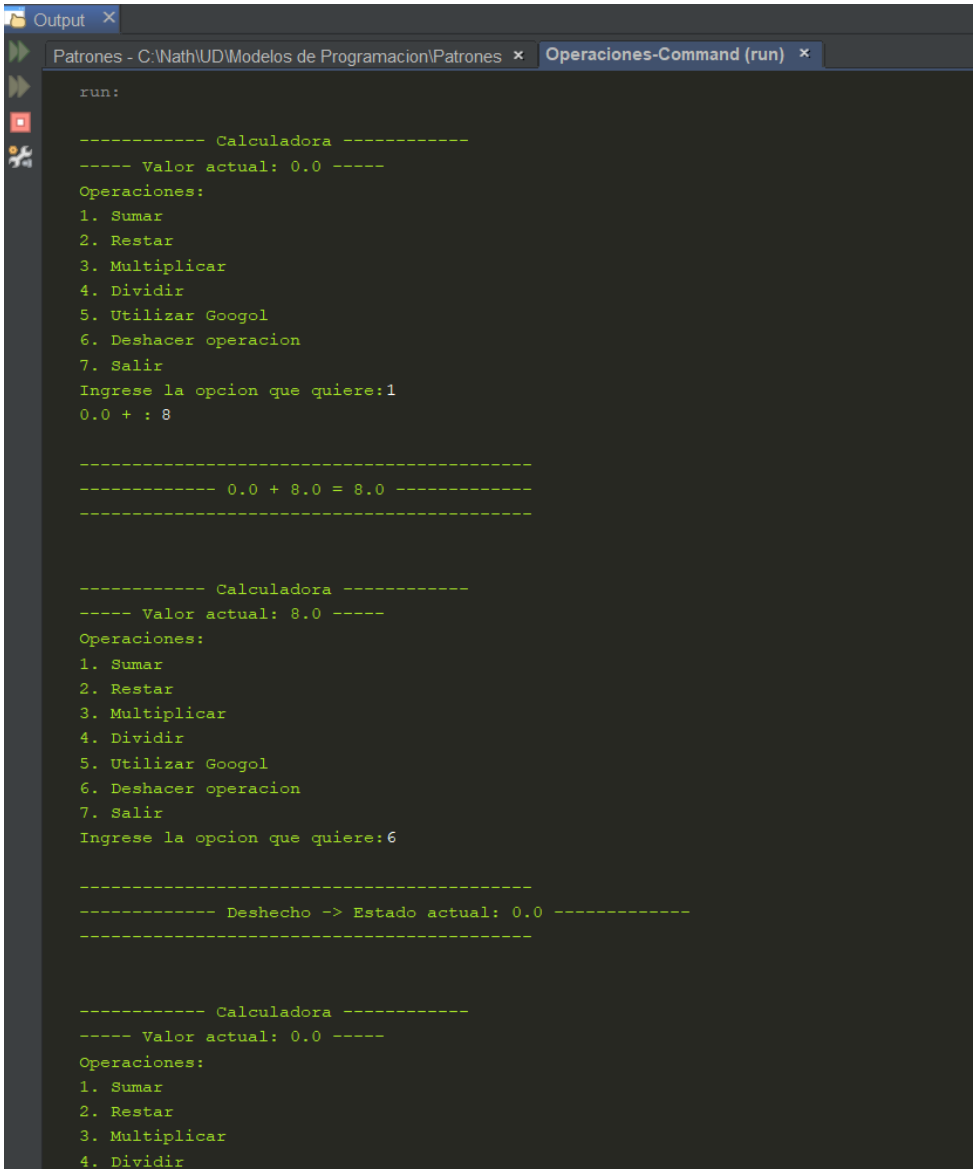


Figura 2: Captura de pantalla de la ejecución en consola.

Pruebas Unitarias

Por cada operacion se hizo una prueba unitaria como la siguiente:

```
1 /**
2  *
3  * @author natha
4  */
```

```java
public class PruebaDividir {

    public static void main(String[]Args) {
        System.out.println("---␣PRUEBA␣␣Operacion␣Dividir␣---");
        Calculadora calc = new Calculadora();
        calc.setActual(50.0);
        OperacionDividir dividir = new OperacionDividir(calc);

        double resultado = dividir.operar(calc.getActual(), 2.0);
        System.out.println("Ejecutar␣operar(50.0,␣2.0)␣->␣
           Resultado:␣" + resultado + "\n|␣Calculadora␣actual:␣" +
            calc.getActual());

        double deshecho = dividir.deshacer();
        System.out.println("\nEjecutar␣deshacer()␣->␣Estado␣
           restaurado␣en␣calculadora:␣" + deshecho);


    }

}
```

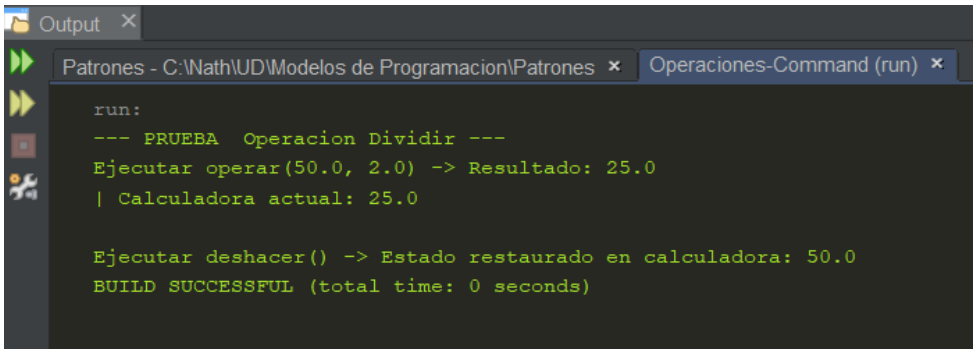


Figura 3: Captura de pantalla de la ejecución en consola.

Repositorio

Enlace GitHub

- `https://github.com/Thalyhx/Modelos-de-Programacion/tree/main/Patrones%20Comportamiento/Operaciones-Command`

Estructura de Paquetes

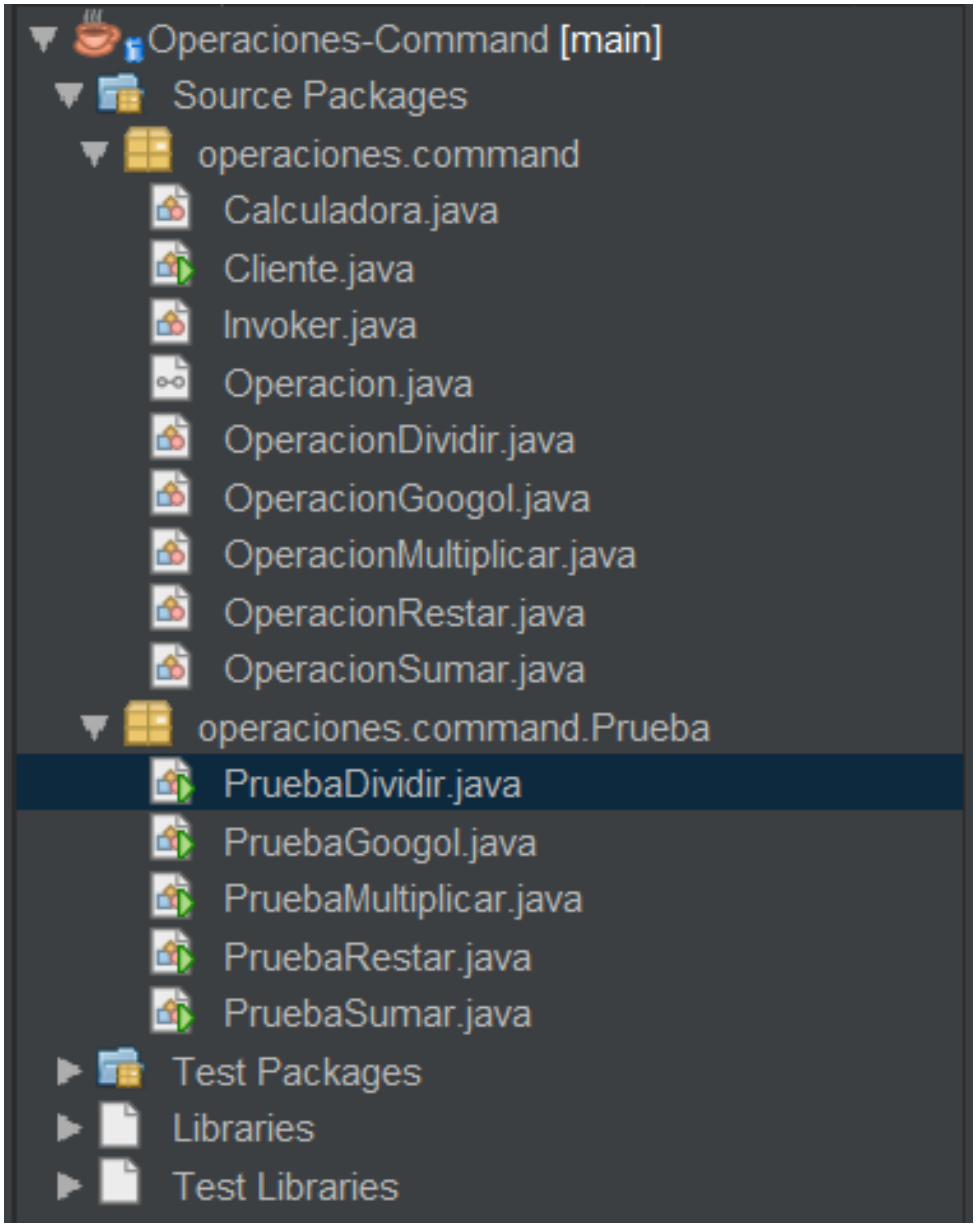


Figura 4: Paquetes y clases.